SEMINAR REPORT

on

HomeStock: A Web-Based Household Inventory Management System

submitted in partial fulfillment

for the award of the Degree of

Bachelor of Technology in

Computer Science & Engineering

Seminar Coordinator: [Name and Designation]

Submitted by: [Student Name] | Roll No.: [Roll Number]

Department of Computer Science & Engineering

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Rajasthan Technical University, Kota

Session 2026-2027

CERTIFICATE

This is to certify that [Student Name], a student of B.Tech. (Computer Science & Engineering), [Semester], has submitted the seminar report entitled “HomeStock: A Web-Based Household Inventory Management System” under my guidance, in partial fulfillment of the requirements for the degree of Bachelor of Technology.

The project description and implementation summary in this report are based on the HomeStock source repository and its accompanying documentation.

[Seminar Coordinator Name]	[Head of Department]
Designation: [Designation]	Department of CSE
Date: [Date]	Place: Jaipur

Replace bracketed fields and obtain the required signatures before submission.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Department of Computer Science & Engineering, Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur, for providing the academic setting in which this seminar report was prepared.

I am thankful to [Supervisor/Coordinator Name] for guidance and feedback throughout the preparation of this report. I also acknowledge the authors and maintainers of the technical documentation referenced here, which helped explain the technologies used in HomeStock.

Finally, I thank my family, friends, and everyone who supported me during the preparation of this work.

[Student Name]
[Roll Number]

ABSTRACT

HomeStock is a web-based household inventory management application that helps an individual record possessions and supplies, their quantities, storage locations, and relevant dates. The project combines a static multi-page frontend built with HTML, CSS, and browser JavaScript with a REST-style API built using Node.js and Express. MongoDB, accessed through Mongoose, stores user, inventory, category, and location records. Authentication uses JSON Web Tokens (JWT), while passwords are hashed using bcryptjs.

The application supports account management, inventory create/read/update/delete operations, stock adjustments, categories, locations, dashboard summaries, expiry and warranty tracking, in-app notifications, and downloadable or printable reports. Its primary purpose is to make routine household tracking more organized and to surface records that need attention, such as items at or below their minimum stock level.

This report explains the problem context, reviews the project's technical approach, describes its architecture and workflows, and discusses its applications and current limitations. The review is based on the source code and project documentation; it does not claim user-study or performance-test results.

Keywords: household inventory, stock tracking, MongoDB, Express, JWT, web application

LIST OF FIGURES

Figure 3.1 High-level architecture of HomeStock

LIST OF TABLES

Table 2.1 Qualitative comparison of household inventory approaches

Table 3.1 Main data entities

Table 3.2 Main API route groups

Table 3.3 Principal use cases

Table A.1 API endpoint summary

CONTENTS

Chapter 1 Introduction
1.1 Background
1.2 Problem Statement
1.3 Motivation
1.4 Objectives
1.5 Scope of the Topic
1.6 Organization of the Report
Chapter 2 Literature Review
2.1 Review of Existing Work
2.2 Technical Source Analysis
2.3 Comparative Study
2.4 Research and Product Gaps
Chapter 3 Methodology
3.1 System Architecture
3.2 Proposed Methodology
3.3 Workflow
3.4 Algorithms and Rules
3.5 Implementation and Use Cases
Chapter 4 Applications and Case Studies
Chapter 5 Conclusion
References
Appendix A API Endpoint Summary

CHAPTER 1 INTRODUCTION

1.1 Background

A household contains many kinds of items: food and medicine with expiry dates, appliances with warranty periods, and everyday supplies distributed across cupboards, rooms, and storage areas. Without a consistent record, people may have difficulty remembering what is available, where it is kept, or when it needs attention. HomeStock addresses this record-keeping problem with a browser-based inventory and a personal account.

1.2 Problem Statement

The central problem is the lack of a simple, consolidated way for an individual to track household item descriptions, quantities, locations, minimum stock levels, and important dates. A list that records only item names cannot support stock decisions or expiry and warranty reminders. HomeStock provides structured item records and uses them to generate summaries and alerts.

1.3 Motivation

The project is motivated by practical household needs: reducing reliance on memory, making supplies easier to locate, noticing low quantities before replenishment is needed, and keeping time-sensitive information visible. It also provides a compact example of connecting a browser interface to an authenticated API and a document database.

1.4 Objectives

- Provide a clear interface for recording and maintaining household inventory.
- Organize records by category and storage location.
- Track current quantity against a user-defined minimum stock threshold.
- Record purchase, expiry, and warranty dates where relevant.
- Summarize inventory and surface low-stock, expiry, and warranty alerts.
- Provide CSV exports and printable reports for personal review.
- Restrict inventory and supporting records to the authenticated user.

1.5 Scope of the Topic

The current implementation is a single-user-per-account household inventory system. It includes a static web frontend, Express API, MongoDB persistence, account and profile operations, inventory management, category and location management, in-app alerts, and reports. The project does not currently include shared household workspaces, email delivery, self-service password recovery, item photo uploads, or deployment configuration.

1.6 Organization of the Report

Chapter 1 introduces the context, problem, objectives, and scope. Chapter 2 reviews the project's technical foundations and compares possible inventory approaches. Chapter 3 explains the architecture, workflows, data model, and implemented rules. Chapter 4

discusses illustrative household applications and benefits. Chapter 5 concludes with limitations and future scope. References and an API appendix follow.

CHAPTER 2 LITERATURE REVIEW

2.1 Review of Existing Work

Household inventory can be maintained informally, in a spreadsheet, or through a purpose-built application. Informal lists are easy to start but may not consistently capture quantities, locations, and dates. A spreadsheet can represent structured records and support sorting, but reminders and account-scoped access generally require additional setup. A web application can combine forms, validation, filtering, and rule-based alerts behind a single interface. These are qualitative observations about the approaches, not results of a comparative user or performance study.

HomeStock adopts a browser-based, client-server design. The official Express documentation describes the framework's HTTP routing and middleware model [2]. MongoDB and Mongoose provide document persistence and schema-based application models [3], [4]. JWT defines a compact token format for representing claims between parties [5]. The project uses these technologies to support authentication, API routing, persistence, and user-specific data operations.

2.2 Technical Source Analysis

The technology documentation informs implementation choices rather than proving product effectiveness. Express routes group related operations, while middleware checks bearer tokens before protected handlers execute. Mongoose schemas define fields and validation constraints for persisted records. The JWT standard describes token structure and claims; the application signs tokens with a one-day expiry. Passwords are processed with bcryptjs rather than stored as readable values. These choices provide the project's technical foundation, but a full security assessment and independent validation are outside this report's scope.

2.3 Comparative Study

Approach	Strengths	Limitations for this problem
Memory or handwritten list	Low setup effort; usable without software.	Difficult to search, keep consistent, or use for automatic date and stock alerts.
Spreadsheet	Flexible columns; familiar sorting and filtering.	Users must maintain formulas and reminders; access and editing workflows are separate.
HomeStock web application	Structured records, account-based API access, filters, stock rules, date tracking, and reports in one application.	Requires a running API and MongoDB; several integrations and collaboration features are not implemented.

Table 2.1: Qualitative comparison of household inventory approaches.

2.4 Research and Product Gaps

The repository provides an implemented application but does not include published usability research, performance benchmarks, or a controlled comparison with other products. The following gaps are therefore project scope opportunities rather than research findings:

- Multi-person household sharing, roles, and synchronization across devices.
- Email or push delivery for alerts and a self-service password-reset workflow.
- Item images, barcode scanning, or assisted data entry.
- More extensive search, configurable alert windows, and richer report filters.
- Production deployment, operational monitoring, backup, and recovery procedures.
- Documented automated tests and user feedback to evaluate correctness and usability.

CHAPTER 3 METHODOLOGY

3.1 System Architecture

HomeStock uses a three-part logical architecture. The browser renders static HTML and CSS and runs JavaScript for page behavior. JavaScript sends JSON requests to the Express API; protected requests include a JWT bearer token. The API validates requests, applies ownership filters, and uses Mongoose to read or update MongoDB documents.

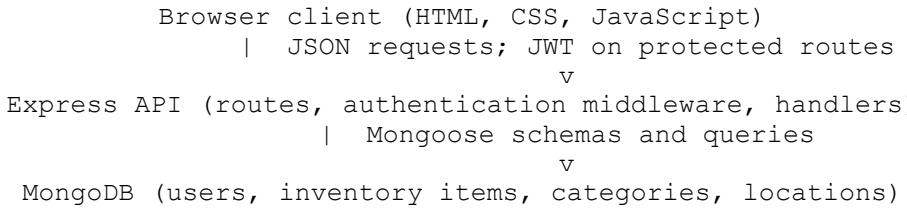


Figure 3.1: High-level architecture of HomeStock.

3.2 Proposed Methodology

The implementation follows a request-response workflow. Each application page identifies its page type, loads shared browser behavior, and requests the records it needs. The API separates routes by feature area and applies authentication middleware to private data operations. MongoDB stores records with an owner reference, and route handlers include that owner in queries so a signed-in user operates on their own records.

3.3 Workflow

1. The user registers or signs in from the authentication pages.
2. The server validates credentials and returns a signed JWT after successful login.
3. The browser stores the token in local storage or session storage according to the remember-me choice.
4. A protected page sends the token in the Authorization header and loads the user's data.
5. The user adds or updates an item, category, or location; the API validates and persists it.
6. Dashboard and alert pages derive counts and reminders from the saved inventory.
7. The user exports selected data as CSV or opens a print-friendly report.

3.4 Algorithms and Rules

Stock status is derived from quantity and minimumStock: quantity at or below zero is out of stock; a positive quantity at or below the threshold is low stock; otherwise the item is in stock. The stock adjustment endpoint accepts a non-zero whole-number change and rejects a resulting negative quantity.

Notification generation scans the authenticated user’s inventory. It creates a low-stock notification when quantity is at or below the threshold, an expiry notification when the expiry date is within the next seven days, and a warranty notification when the warranty date is within the next thirty days. Notification read state and alert preferences are saved in that browser’s local storage; they are not synchronized to the server.

The dashboard counts inventory records and distinct category and location names, and shows recent additions, low-stock records, and items expiring within fourteen days. The dedicated expiry page uses its own date filters, including expired items and dates within thirty days. The warranty page summarizes records by warranty date.

3.5 Implementation and Use Cases

Entity	Purpose and principal fields
User	Account identity: first name, last name, normalized unique email, hashed password.
Inventory	User-owned item: name, category, location, quantity, minimum stock, purchase/expiry/warranty dates, notes.
Category	User-owned grouping: name, normalized name key, icon, description.
Location	User-owned storage place: name and icon.

Table 3.1: Main data entities in the application.

Route group	Principal operations
/api/auth	Registration, login, profile, password change, account deletion.
/api/inventory	List, create, update, delete, and adjust item stock.
/api/categories	List, create, and delete categories.
/api/locations	List, create, and delete locations.
/api/notifications	Generate in-app low-stock, expiry, and warranty alerts.

Table 3.2: Main API route groups.

Use case	User action	Application response
Maintain supplies	Create or edit an item and set quantity and minimum stock.	Persist the item and display its current stock status.
Find an item	Search or filter by name, category, location, or status.	Show matching records in the inventory list.
Review expiry	Open expiry tracker and inspect date-based results.	Show expired and upcoming records; allow quantity reduction for use/disposal.
Prepare a report	Choose a report and CSV or print action.	Download CSV data or open a print-friendly report.

Table 3.3: Principal HomeStock use cases.

CHAPTER 4 APPLICATIONS AND CASE STUDIES

4.1 Applications

HomeStock is intended for personal household record keeping. It can be used to track pantry and cleaning supplies, medicine with expiry dates, appliances with warranty dates, or stored belongings grouped by room. Its account model is personal; shared editing by multiple household members is not currently provided.

4.2 Illustrative Use Scenarios

Scenario 1: Pantry and household supplies

A user records packaged food and cleaning products with quantity, a minimum stock threshold, kitchen or storage location, and expiry date. The stock page helps identify low quantities, while the expiry tracker highlights items approaching their recorded date. The user can export a CSV to review or use as a replenishment list.

Scenario 2: Appliances and warranty dates

A user records an appliance, where it is stored, its purchase date, and its warranty end date. The warranty page provides a chronological view and the notification service surfaces dates approaching within its configured window. The record remains part of the user's inventory.

These scenarios illustrate how the implemented features may be used. They are not reports of observed customer deployments or measured outcomes.

4.3 Benefits and Impact

- Centralizes item quantities, categories, locations, and relevant dates.
- Makes stock and date conditions visible through lists, dashboard summaries, and alerts.
- Supports data review outside the application through CSV and printable reports.
- Demonstrates separation between a browser client, an API, and persistent storage.
- Provides per-user record scoping for the application's account-based use.

4.4 Operational Considerations

The app requires the frontend to be served separately from the API and needs a reachable MongoDB instance. Production use would require deployment configuration, HTTPS, appropriate CORS origin restrictions, secret management, database backups, monitoring, and operational procedures. These deployment elements are not supplied as a completed production setup in the project documentation.

CHAPTER 5 CONCLUSION

5.1 Conclusion

HomeStock brings together common household inventory tasks in a web application: recording items, quantities, categories, storage places, and important dates; reviewing stock and date-based alerts; and exporting summaries. Its static browser frontend communicates with an authenticated Express API, which persists user-scoped data in MongoDB through Mongoose. The implementation demonstrates a practical full-stack application organized around everyday record keeping.

5.2 Takeaways

- Household inventory records are more useful when quantity, location, and dates are maintained together.
- Threshold and date rules can transform stored records into actionable reminders.
- User ownership must be applied consistently to API operations, not only to frontend navigation.
- Exports and printable views provide a simple way to use information beyond the app.
- The present implementation is a useful foundation but has clear integration and deployment gaps.

5.3 Future Scope

- Add shared household spaces with invitations, membership, and role-based permissions.
- Implement verified password reset and configurable email or push notifications.
- Add item images and optional barcode-based entry.
- Provide pagination, richer server-side search, configurable alert windows, and report filters.
- Add automated API and browser tests and document test coverage.
- Prepare production deployment guidance, restricted CORS settings, HTTPS, backups, monitoring, and recovery plans.
- Evaluate usability with representative users and measure performance with documented datasets and methods.

REFERENCES

[1] Node.js, “Node.js Documentation.” Available: https://nodejs.org/docs/latest/api/

[2] Express.js, “Express 5.x API Reference.” Available: https://expressjs.com/en/5x/api.html

[3] MongoDB, “MongoDB Manual.” Available: https://www.mongodb.com/docs/manual/

[4] Automattic, “Mongoose Documentation.” Available: https://mongoosejs.com/docs/

[5] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7519

[6] OWASP Foundation, “Password Storage Cheat Sheet.” Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[7] HomeStock project source code and project documentation, local repository, accessed 31 October 2026.

APPENDIX A API ENDPOINT SUMMARY

The following endpoints are documented by the project. Routes other than registration, login, root, and health require authentication as described in the API README.

Method	Endpoint	Purpose
POST	/api/auth/register	Create an account.
POST	/api/auth/login	Authenticate and receive a JWT.
GET	/api/auth/profile	Read the current user's profile.
PATCH	/api/auth/profile	Update the current user's name.
PATCH	/api/auth/password	Change password after current-password verification.
DELETE	/api/auth/profile	Delete the account and its inventory/category/location data.
GET / POST	/api/inventory	List or create inventory items.
PATCH / DELETE	/api/inventory/:id	Update or delete an inventory item.
PATCH	/api/inventory/:id/stock	Adjust item quantity.
GET / POST	/api/categories	List or create categories.
DELETE	/api/categories/:id	Delete an unused category.
GET / POST	/api/locations	List or create locations.
DELETE	/api/locations/:id	Delete an unused location.
GET	/api/notifications	Generate alerts from current inventory.
GET	/health	Check API process health.

Table A.1: API endpoint summary.

Local API configuration uses MONGO_URI and JWT_SECRET in server/.env. The repository's .env.example contains placeholders; actual secrets must remain private.